Code and Data Replication Package: *Matching Technology, Fleet Size, and Passenger Wait in Ride-Hailing Markets*

Authors: Kaitlin M. Daniels and Danko Turcic
Journal: *Management Science*
Public GitHub Repository: https://github.com/dxt48/Matching-Technologies-in-Ride-Hailing-Markets.git

Document Definitions

Throughout this guide and in the accompanying code comments, we use the following shorthand definitions to refer to the manuscript files:

- `(Main_Document)`: Refers to the main body of the paper (`paper_minor_2.tex` / `paper_minor_2.pdf`, Sections 1–6).
- `(Appendix)`: Refers to the Electronic Companion / Online Appendix (`EC_appendix_minor_2.tex`, Sections EC.1–EC.3).

Data and Code Disclosure & Provenance Statement

1. **Public Availability of Code and Data**:
   - All simulation code, analysis scripts, pre-computed CSV datasets, and generated figures included in this replication package are publicly available without restriction (and are also hosted at https://github.com/dxt48/Matching-Technologies-in-Ride-Hailing-Markets.git, as cited in Section 5.1 of `(Main_Document)`).
2. **Source of the CSV Data Files (`Data/`)**:
   - All 6 `.csv` files in the `Data/` directory are synthetic agent-based simulation outputs generated directly by the Python simulation scripts in `Code/` (`SH_10.py`, `CD_10.py`, and `CD_10_Batch.py`). No proprietary, confidential, or restricted-access datasets are used.
3. **Source of Aggregate Calibration Parameters (Table 1 in `(Main_Document)`)**:
   - As described in Section 5.1 of `(Main_Document)`, the four aggregate city primitives used to calibrate the simulation—grid size ($J = 80 \times 80 = 6,400$ corners representing a 16-square-mile area of Manhattan south of 128th Street), average vehicle speed (11 mph), average trip duration (12 minutes), and aggregate passenger request rate ($23,592$ requests per hour)—are taken from publicly available New York City Taxi and Limousine Commission (TLC) trip records (October–November 2011; https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page) and the published structural estimates of Fréchette, Lizzeri, and Salz (*American Economic Review*, 2019, Section II.B).

Important Note on How This Package Is Structured (Two Ways to Run)

The code in this replication package is designed to operate in **two distinct modes** as a time-saving feature for reviewers and auditors:

1. **Mode 1 — Non-Destructive Audit & Reproduction from Existing Data (Default, takes < 10 seconds):**
   - Because the simulation CSV data files (`Data/`) and the reference figure files (`Figures/`) are already provided in this package, running `python run_all.py` (or running `Code/Master_Analysis.py` and `Code/Lem_4_Prop_3.py` individually) reads the existing CSV data directly, executes all statistical estimations and hypothesis tests, prints the exact tables reported in `(Main_Document)` and `(Appendix)`, and **draws and displays (`plt.show()`) all figures without overwriting the existing PNG files in `Figures/`** (`SAVE_FIGURES = False` by default).
   - Similarly, the simulation scripts (`Code/SH_10.py`, `Code/CD_10.py`, and `Code/CD_10_Batch.py`) check whether the CSV files already exist in `Data/` and, by default (`FORCE_RERUN_SCALING = False` and `FORCE_RERUN_WAITS = False`), load the existing data rather than re-running the simulations and overwriting the CSV files.
2. **Mode 2 — Generating the Simulation Data from Scratch (Takes Many Hours):**
   - The files `Code/SH_10.py`, `Code/CD_10.py`, and `Code/CD_10_Batch.py` contain the discrete-time, agent-based simulation engines. The `_10` in their filenames denotes that each script executes **10 independent simulation shifts (replications)** for every single fleet size in the scaling sweep (23 to 24 fleet-size levels $\times$ 10 shifts = 230 to 240 full city shifts per matching technology), plus 10 extended shifts (2,000 periods each, generating over 2 million individual passenger trips per technology) to capture the full wait-time distributions.
   - Because tracking thousands of individual drivers and hundreds of thousands of passenger queues across an 80×80 grid over 10 independent shifts is computationally intensive, **regenerating the CSV data from scratch takes many hours** depending on computer speed.
   - For this reason, we provide all pre-computed CSV outputs in `Data/` so reviewers can immediately audit the statistical analyses and figures without waiting hours for the simulations to finish, while still providing the complete simulation engines for full transparency.

Part 1: Overview of the Analysis, Figures, Tables, and Data

1.1 Summary of the Empirical and Simulation Analysis

The analytical model in Section 4 of `(Main_Document)` develops closed-form approximations for the steady-state distribution and expected value of passenger waiting times under two canonical ride-hailing matching technologies:

1. **Street-Hailing (SH)** (Approximation 1 in Section 4.1 of `(Main_Document)`): Drivers search via a random walk on a wrap-around (toroidal) grid and pick up waiting passengers upon encountering them at a corner. In the *matching-dominated regime*, expected passenger wait scales inversely with fleet size ($\bar{W}_s \propto (n_s - \beta_s)^{-1}$, scaling exponent $\delta = 1.0$) and positive passenger waiting times follow an **Exponential** distribution (Weibull shape parameter $k = 1.0$).
2. **Central Dispatch (CD)** (Approximation 2 in Section 4.2 of `(Main_Document)`): A central platform matches arriving passengers to the nearest idle driver (or matches accumulated requests in short time windows under batched central dispatch) and routes drivers along shortest Manhattan paths. In the *en-route-*

dominated regime, expected passenger wait scales inversely with the square root of fleet size ($\bar{W}_d \propto (n_d - \beta_d)^{-0.5}$, scaling exponent $\delta = 0.5$) and positive passenger waiting times follow a **Rayleigh** distribution (Weibull shape parameter $k = 2.0$).

To validate these predictions and evaluate their managerial implications (Sections 5 and 6 of `(Main_Document)` and Sections EC.2 and EC.3 of `(Appendix)`), the code performs four sets of analyses:

1. **10-Shift Agent-Based Grid Simulations (`Code/SH_10.py`, `Code/CD_10.py`, `Code/CD_10_Batch.py`)**:
   - Simulates an 80×80 wrap-around grid ($J = 6,400$ corners) across $N = 10$ independent random seeds (`0` through `9`, representing 10 distinct shifts).
   - **Cold Start and 1-Hour Burn-In**: Each simulation shift begins with a "cold start" (no waiting passengers in the grid and all drivers initially idle). Because each discrete period represents 0.3 minutes (18 seconds), the first `burn_in = 200` periods ($200 \times 0.3 = 60$ minutes, or **1 full hour**) of every shift are discarded as a burn-in period so the system reaches steady state before any passenger waiting-time statistics are recorded.
   - **Scaling Sweeps**: Evaluates post-burn-in mean passenger waiting times across fleet sizes $n_s \in \{6,400, 6,500, \dots, 8,700\}$ for Street-Hailing and $n_d \in \{5,300, 5,400, \dots, 7,500\}$ for standard and batched Central Dispatch.
   - **Distributional Deep Dives**: Collects post-burn-in individual passenger waiting times across 10 shifts (over 2 million served passengers per matching technology) to evaluate the full wait-time distribution.
2. **Average Wait Scaling Validation & AIC Regime Tournament (`Code/Master_Analysis.py`)**:
   - Estimates the log-linearized scaling regression $\ln \bar{W}_i = \gamma - \delta \ln(n_i - \beta) + \epsilon_i$ (Equation (3) in Section 5.2.1 of `(Main_Document)`).
   - Conducts an Akaike Information Criterion (AIC) "Regime Tournament" comparing the constrained theoretical scaling exponents ($\delta = 1$ vs. $\delta = 0.5$) and estimates the free scaling exponent $\hat{\delta}$ and asymptote $\hat{\beta}$.
3. **Wait-Time Distribution Structural Recovery (`Code/Master_Analysis.py`)**:
   - Separates the discrete zero-wait point mass ($W = 0$, immediate corner pickups) from positive travel delays ($W > 0$).
   - Fits a Weibull distribution to $W > 0$ to test family recovery ($k = 1$ Exponential vs. $k = 2$ Rayleigh) and computes quantile-quantile (Q-Q) plots and body linearity (R^2 over the 0–95% quantile body).
4. **Capacity Premium & Short-Wait Advantage / Tail-Risk Calibration (`Code/Lem_4_Prop_3.py` and `Code/Theory.py`)**:
   - Tests Lemma 5 (Capacity Premium) and Proposition 1 (Short-Wait Advantage / CDF Crossing) in Section 6.2 of `(Main_Document)` and Section EC.3 of `(Appendix)`.
   - Uses a non-parametric bootstrap across the 10 independent simulation shifts to calibrate the fleet sizes $n_s^$ and $n_d^$ required to equate expected wait at $W^* = 1.5$ minutes, and compares the resulting CDFs $F_s(w, \lambda, n_s^)$ and $F_d(w, \lambda, n_d^)$.

1.2 Mapping of Figures to Code and Data

(Note: Figures 1 and 2 in `(Main_Document)` are static conceptual illustrations and inline LaTeX TikZ diagrams, respectively, and do not use simulation data. Figures 3–6 are generated by the Python scripts in this package.)

Figure in `(Main_Document)`	Title / Description	Output Image File	Generating Script	Input CSV Data Files
Figure 1 (p. 4)	*Model of a City* (Panels a & b)	`Figures/Figure Example City.PNG`, `Figures/central dispatch figure 2.pdf`	N/A (Conceptual diagram)	None
Figure 2 (p. 6)	*Illustration of Operating Regimes for Central Dispatch and Street-Hailing*	Inline TikZ in `(Main_Document)`	N/A (Inline LaTeX TikZ)	None
Figure 3 (p. 15)	*Street-Hailing: Structural Validity* • Panel A: Scaling Validation • Panel B: Structural Recovery ($W>0$)	`Figures/SH_Structural_Analysis.png`	`Code/Master_Analysis.py`	• Panel A: `Data/sim_scaling_S` • Panel B: `Data/sim_waits_SH.`

Figure in (Main_Document)	Title / Description	Output Image File	Generating Script	Input CSV Data Files
Figure 4 (p. 15)	*Central Dispatch: Structural Validity* • Panel A: Scaling Validation • Panel B: Structural Recovery ($W>0$)	Figures/CD_Structural_Analysis.png	Code/Master_Analysis.py	• Panel A: Data/sim_scaling_C • Panel B: Data/sim_waits_CD.
Figure 5 (p. 16)	*Central Dispatch with Batching: Structural Validity* (0.6-min batch window) • Panel A: Scaling Validation • Panel B: Structural Recovery ($W>0$)	Figures/CD_BATCHED_Structural_Analysis.png	Code/Master_Analysis.py	• Panel A: Data/sim_scaling_CD_batche • Panel B: Data/sim_waits_CD_batched_
Figure 6 (p. 18)	*Comparison of Wait Time Distributions ($W^ \approx 1.9$ Min)*	Figures/wait_time_comparison.png	Code/Lem_4_Prop_3.py (with Code/Theory.py)	None (Analytical CDF evaluation a $n_s^, *n_d^*$ for $W^* = 1.5$ min, at $w' \approx 1.91$ min)

1.3 Mapping of Tables and In-Text Statistics to Code and Data

Table / Section	Location	Statistics Reported	Generating Script	Input CSV Data Files
Table 1 (tab:SiMparams)	(Main_Document), Section 5.1	Calibration primitives ($J=6,400$, speed 11 mph, trip duration 12 min, arrival rate $23,592$/hr)	Calibrated from NYC TLC data & Fréchette et al. (2019)	N/A
Table 2 (tab:regime_bounds)	(Main_Document), Section 5.1	Analytical regime boundaries: • CD: $n_{\text{base}}=4,719$, $n_{\min}=5,359$, $n_{\max}=7,654$ • SH: $n_{\text{base}}=5,800$, $n_{\max}=8,735$	Closed-form formulas in Section 5.1 of (Main_Document) (see note below)	N/A

Table / Section	Location	Statistics Reported	Generating Script	Input CSV Data Files
Table 3 (`tab:scaling_audit_combined`) & text in Sections 5.2.1–5.2.2 and `(Appendix)` Section EC.2.3	`(Main_Document)`, Section 5.2; `(Appendix)`, Section EC.2.3	• **Panel A (AIC Tournament):** Evidence ratios ($1:4.2\times 10^{48}$ SH, $1:6.1\times 10^{41}$ CD, $1:3.6\times 10^{60}$ Batched) and R^2 ($0.9814, 0.9796, 0.9790$) • **Panel B (Free Exponent $\hat{\delta}$):** 1.014 $[0.995, 1.034]$ (SH), 0.514 $[0.504, 0.524]$ (CD), 0.481 $[0.472, 0.491]$ (Batched) • **Panel C (Asymptote $\hat{\beta}$):** $5,207$ (SH), $5,244$ (CD), $5,151$ (Batched) • **Panel D (Distribution Recovery):** $% W=0$ ($43.4%, 16.5%, 8.5%$), Weibull $\hat{k}$ ($1.08, 1.80, 1.90$), Q-Q Body R^2 ($0.9938, 0.8388, 0.8973$)	`Code/Master_Analysis.py` (prints full LaTeX table to console)	• `Data/sim_scaling_SH.csv` • `Data/sim_waits_SH.csv` • `Data/sim_scaling_CD.csv` • `Data/sim_waits_CD.csv` • `Data/sim_scaling_CD_batche` • `Data/sim_waits_CD_batched`
Table EC.1 (`tab:prop3_test`) & text in Section 6.2 and `(Appendix)` Section EC.3	`(Main_Document)`, Section 6.2; `(Appendix)`, Section EC.3	• Bootstrap calibrated fleet sizes for $W^{=1.5}$ *min across 10 shifts:* $n_s^ = 7,012$ (95% CI $[7,002, 7,023]$) and $n_d^* = 5,296$ (95% CI $[5,287, 5,305]$), a $\approx 32%$ capacity premium • Short-wait probabilities $P(W \le 1.5)$: 0.6321 ($63.2%$) for SH vs. 0.5441 ($54.4%$) for CD (difference $+0.0881$, or $16.2%$ lower under CD)	`Code/Lem_4_Prop_3.py` (`perform_statistical_test()`)	• `Data/sim_scaling_SH.csv` • `Data/sim_scaling_CD.csv`

- **Arithmetic Breakdown of Analytical Bounds in Table 2 (`tab:regime_bounds`)**:
  - **Stability Lower Bound (n_{base})**: From Table 1, aggregate demand is $\lambda J = 23,592\text{ rides/hr} = 393.2\text{ rides/min}$ and average trip duration is $1/(\tau \nu) = 12\text{ min}$. For Central Dispatch, $n_{\text{base}} = \frac{\lambda J}{\tau \nu} = 393.2 \times 12 = 4,718.4 \approx \mathbf{4,719}$. For Street-Hailing, $n_{\text{base}} = \frac{\lambda J (1 + \tau)}{\tau \nu}$ (reported as $\mathbf{5,800}$ in Table 2 under the baseline block specification and $\approx \mathbf{4,836}$ in Table 3 under $1/\tau = 40$ simulation blocks).
  - **Heavy-Traffic Lower Bound ($n_{\min}$ for CD)**: Ensures driver utilization does not exceed 90% ($\ge 10%$ of corners have an idle driver): $n_{\min} = n_{\text{base}} + 0.10 \times J = 4,719 + 640 = \mathbf{5,359}$.
  - **Saturation Upper Bound ($n_{\max}$)**: Defined via the classical occupancy problem so the probability a passenger arrives at a corner already occupied by an idle driver ($1 - (1 - 1/J)^{\tilde{n}} \approx 1 - e^{-\tilde{n}/J}$) does not exceed $e^{-1} \approx 0.3679$, which corresponds to $\tilde{n} = -J \ln(1 - e^{-1}) = 6,400 \times 0.45867 = 2,935.5$ idle drivers above n_{base}: $4,719 + 2,935 = \mathbf{7,654}$ (Central Dispatch) and $5,800 + 2,935 = \mathbf{8,735}$ (Street-Hailing).

1.4 Data Dictionary (`Data/` Directory)

All pre-computed simulation outputs from the 10-shift runs are stored in `Data/` as comma-separated values (`.csv`) files:

1. **Scaling Sweep Files**:
   - `Data/sim_scaling_SH.csv` (240 rows: 10 shifts/seeds $\times$ 24 fleet sizes $n_s \in [6400, 8700]$)
   - `Data/sim_scaling_CD.csv` (230 rows: 10 shifts/seeds $\times$ 23 fleet sizes $n_d \in [5300, 7500]$)
   - `Data/sim_scaling_CD_batched_w2.csv` (230 rows: 10 shifts/seeds $\times$ 23 fleet sizes $n_d \in [5300, 7500]$, batch window = 2 periods = 0.6 min)
   - **Columns**:

    - `seed` (`int`): Simulation shift / random number generator seed (`0` through `9`).
    - `n_s` or `n_d` (`int`): Total number of active drivers in the city.
    - `wait` (`float`): Post-burn-in mean passenger waiting time in minutes for that shift and fleet size.

2. **Individual Wait-Time Distribution Files**:

    - `Data/sim_waits_SH.csv` (2,094,887 individual passenger records across 10 shifts)
    - `Data/sim_waits_CD.csv` (2,125,568 individual passenger records across 10 shifts)
    - `Data/sim_waits_CD_batched_w2.csv` (2,125,179 individual passenger records across 10 shifts with batch window = 2 periods = 0.6 min)
    - **Columns**:
        - `seed` (`int`): Simulation shift / random number generator seed (`0` through `9`).
        - `wait_min` (`float`): Total waiting time (match delay + en-route pickup time) in minutes for an individual served passenger after the 1-hour burn-in period.

Part 2: Code Description, Computing Environment, and Execution Instructions

2.1 Description of Code Files

- **`run_all.py`** (Root Directory):
    - **Purpose**: One-click master replication script. Verifies that all 6 pre-computed CSV files exist in `Data/`, runs `Code/SH_10.py`, `Code/CD_10.py`, and `Code/CD_10_Batch.py` in load-only verification mode, and then executes `Code/Master_Analysis.py` and `Code/Lem_4_Prop_3.py` to print **Tables 3 and EC.1** and display **Figures 3, 4, 5, and 6** (`plt.show()`) without overwriting any existing files in `Data/` or `Figures/`.
- **`replication_output.txt`** (Root Directory):
    - **Purpose**: Pre-saved console output log from running `python run_all.py`, allowing reviewers to immediately inspect the exact numerical and LaTeX outputs for **Table 3** and **Table EC.1**.

A. Analysis and Figure Generation Scripts (`Code/`)

- **`Code/Master_Analysis.py`**:
    - **Purpose**: Reads the 6 CSV files in `Data/`, performs the log-linearized non-linear least squares scaling regressions (`scipy.optimize.curve_fit`), runs the AIC regime classification tournament, conducts the Weibull structural recovery test (`scipy.stats.weibull_min.fit`) and Q-Q linearity calculations on positive waits ($W > 0$), draws and displays **Figures 3, 4, and 5** via `plt.show()` (with `SAVE_FIGURES = False` by default so the provided PNG files in `Figures/` are not overwritten), and prints the exact LaTeX code for **Table 3** (`tab:scaling_audit_combined`) to standard output.
- **`Code/Lem_4_Prop_3.py`**:
    - **Purpose**: Performs two tasks for Section 6.2 of `(Main_Document)` and Section EC.3 of `(Appendix)`:
        1. `perform_statistical_test()`: Reads `Data/sim_scaling_SH.csv` and `Data/sim_scaling_CD.csv`, fits the scaling curves shift-by-shift across the 10 simulation seeds, computes the non-parametric bootstrap point estimates and 95% t-confidence intervals for the fleet sizes $(n_s^, *n_d^*)$ that achieve target mean wait $W^* = 1.5$ minutes, and prints all numbers reported in **Table EC.1** (`tab:prop3_test`).
        2. `plot_tail_prob_illustration()`: Uses the theoretical functions in `Code/Theory.py` to solve for the fleet sizes equating analytical expected waits at $W^* = 1.5$ minutes, identifies the unique CDF crossing threshold $w' \approx 1.91$ minutes (Proposition 1), and draws and displays **Figure 6** via `plt.show()` (`SAVE_FIGURES = False` by default so `Figures/wait_time_comparison.png` is not overwritten).
- **`Code/Theory.py`**:
    - **Purpose**: Helper module imported by `Code/Lem_4_Prop_3.py`. Implements the closed-form PDF, CDF, and expected wait expressions from **Approximation 1** (`f_s`, `F_s`, `taxi_wait`) and **Approximation 2** (`f_d`, `F_d`, `uber_wait`) in Section 4 of `(Main_Document)`.

B. 10-Shift Simulation Data Generation Scripts (`Code/`)

The `_10` suffix in the filenames below indicates that each script runs **10 independent simulation shifts** (`NUM_SEEDS = 10`, seeds `0` through `9`) for every fleet-size configuration:

- **`Code/SH_10.py`**:
    - **Purpose**: Implements **Algorithm 1** (*Toroidal Street Hailing Simulation with Queue Dynamics*, Section EC.2.1 of `(Appendix)`). Runs Loop 1 (10 shifts across 24 fleet sizes n_s) to produce `Data/sim_scaling_SH.csv` and Loop 2 (10 extended shifts of 2,000 periods each collecting individual passenger waits) to produce `Data/sim_waits_SH.csv`.
- **`Code/CD_10.py`**:
    - **Purpose**: Implements **Algorithm 2** (*Centralized Dispatch Simulation with Global Greedy Matching*, Section EC.2.2 of `(Appendix)`). Runs Loop 1 (10 shifts across 23 fleet sizes n_d) to produce `Data/sim_scaling_CD.csv` and Loop 2 (10 extended shifts of 2,000 periods each collecting individual passenger waits) to produce `Data/sim_waits_CD.csv`.
- **`Code/CD_10_Batch.py`**:
    - **Purpose**: Implements the batched central dispatch robustness check described in Section EC.2.3 of `(Appendix)`, where passenger requests are accumulated and matched every `BATCH_WINDOW = 2` periods (0.6 minutes). Runs 10 shifts across 23 fleet sizes to produce `Data/sim_scaling_CD_batched_w2.csv` and 10 extended shifts to produce `Data/sim_waits_CD_batched_w2.csv`.

2.2 Software Requirements, Computing Environment, and Expected Runtimes

- **Operating System**: Cross-platform (tested on macOS, Linux, and Windows).
- **Hardware Requirements**: Standard desktop or laptop CPU, ≥ 4 GB RAM, ~ 60 MB disk space.
- **Python & Package Versions**: Python 3.9+ (tested with Python 3.14.5, `numpy==2.4.6`, `pandas==3.0.3`, `scipy==1.17.1`, `matplotlib==3.10.9`, `statsmodels==0.14.6`).

To install all required packages using the provided `requirements.txt`:

```
pip install -r requirements.txt
```

Execution Mode	Script(s)	Preserves Existing `Data/` & `Figures/`?	Approximate Runtime
Mode 1: Non-Destructive Audit & Reproduction (Default)	`python run_all.py` *(or `Code/Master_Analysis.py` + `Code/Lem_4_Prop_3.py`)*	**Yes** (`FORCE_RERUN_* = False`, `SAVE_FIGURES = False`)	**~5–15 seconds**
Mode 2: Regenerate Simulation Data from Scratch	`Code/SH_10.py`, `Code/CD_10.py`, `Code/CD_10_Batch.py`	**No** (Requires flipping `FORCE_RERUN_* = True`)	**~2–6 hours total** (240–250 shifts per script)

2.3 Package Directory Structure and How to Run the Code

```
Empirical_Analysis_Code_MNSC/
├── README.md
├── requirements.txt
├── run_all.py
├── replication_output.txt
├── Code/
│   ├── Master_Analysis.py
│   ├── Lem_4_Prop_3.py
│   ├── Theory.py
│   ├── SH_10.py
│   ├── CD_10.py
│   └── CD_10_Batch.py
├── Data/
│   ├── sim_scaling_SH.csv
│   ├── sim_waits_SH.csv
│   ├── sim_scaling_CD.csv
│   ├── sim_waits_CD.csv
│   ├── sim_scaling_CD_batched_w2.csv
│   └── sim_waits_CD_batched_w2.csv
└── Figures/
    ├── Figure Example City.PNG
    ├── central dispatch figure 2.pdf
    ├── SH_Structural_Analysis.png
    ├── CD_Structural_Analysis.png
    ├── CD_BATCHED_Structural_Analysis.png
    └── wait_time_comparison.png
```

All commands below should be executed from the root of this folder (`Empirical_Analysis_Code_MNSC/`):

Option A (Recommended for Audit): Reproduce All Statistics and Display Figures Without Overwriting Files (< 15 seconds)

To compute all statistics in **Table 3** and **Table EC.1** and draw/display **Figures 3, 4, 5, and 6** (`plt.show()`) using the existing CSV files in `Data/`—while leaving the reference files in `Data/` and `Figures/` completely untouched:

```
python run_all.py
```

Alternatively, you may run the two analysis scripts individually:

1. **Run the Main Scaling & Distributional Analysis (Prints Table 3 & Displays Figures 3, 4, 5):**

   ```
   python Code/Master_Analysis.py
   ```

2. **Run the Capacity Premium & Tail-Risk / Short-Wait Analysis (Prints Table EC.1, Section 6.2 & Displays Figure 6):**

   ```
   python Code/Lem_4_Prop_3.py
   ```

   (Note: If a reviewer wishes to save newly rendered PNG files to disk, they can set `SAVE_FIGURES = True` near the top of `Code/Master_Analysis.py` and `Code/Lem_4_Prop_3.py`.)

Option B: Re-Run the 10-Shift Simulations from Scratch (Warning: Takes Many Hours)

If you wish to regenerate the raw CSV files from scratch by running all 10 shifts across all fleet sizes:

1. Open `Code/SH_10.py`, `Code/CD_10.py`, or `Code/CD_10_Batch.py` in a text editor.
2. Change the two safety flags near the top of the script from `False` to `True`:

```
FORCE_RERUN_SCALING = True  # Set to True to regenerate scaling data (10 shifts per fleet size)
FORCE_RERUN_WAITS = True    # Set to True to regenerate individual wait distribution data (10 shifts)
```

3. Run the script(s) from `Empirical_Analysis_Code_MNSC/`:

```
python Code/SH_10.py
python Code/CD_10.py
python Code/CD_10_Batch.py
```

Please note that each script executes 240–250 full simulation runs of an $80 |times 80$ grid city with thousands of agents and over two million passenger arrivals, which requires several hours of compute time depending on your hardware.